

Consultório VR — Tutorial completo

Servidor Linux (DCC/DGTI) + Meta Quest na rede 5G · Unity 6000.0.39f1 · repositório [danielreissss/Projeto-VR](#)

```
[Notebook] --cabo DCC / ssh--> [Servidor Linux 172.30.22.31] --cabo--> [Roteador 5G]
                                     roda: ConsultorioServer                |
                                     (servidor dedicado, UDP 7777)           | Ethernet -> USB-C
                                     |                                     |
                                     [Meta Quest com o APK]                 |
                                     (cliente: conecta no IP)                |
```

0. Visão geral

- **Óculos (APK):** têm o cenário inteiro dentro do app. Ao abrir, conectam sozinhos no IP do servidor configurado no componente `LanDirectConnect` da cena (`172.30.22.31`).
- **Servidor Linux:** roda o mesmo cenário *sem gráficos* como **servidor dedicado**. Não precisa de IP configurado: escuta em todas as interfaces na porta **UDP 7777**. Cria os avatares de quem conecta, sincroniza posições e faz o relay da voz.
- **Ordem obrigatória no dia:** primeiro o servidor rodando, depois abrir o app nos óculos.

Parte	Onde	Quando
1. Preparar o projeto (git + LFS)	PC de casa (Windows)	Em casa
2. Gerar e instalar o APK nos óculos	PC de casa + Quest no USB	Em casa
3. (Opcional) gerar build do servidor no Windows	PC de casa	Em casa — plano B
4. Preparar o servidor: git, Git LFS, clone, Unity	Servidor Linux via SSH	No lab
5. Gerar o build do servidor no próprio servidor	Servidor Linux	No lab
6. Rodar o servidor	Servidor Linux	No lab
7. Conectar os óculos e entrar	Quest + roteador 5G	No lab

Segurança: a senha do servidor foi enviada por print. Não a coloque em arquivos do repositório nem em commits. Se possível, peça para trocá-la depois.

PARTE 1 — Preparar o projeto em casa (Windows)

1.1 Instalar as ferramentas

1. **Git for Windows:** <https://git-scm.com/download/win> (já inclui o Git LFS).
2. **Unity Hub:** <https://unity.com/download>. Faça login com sua conta Unity.
3. No Hub → **Installs** → **Install Editor** → **Archive** e instale a versão **6000.0.39f1** com os módulos:
 - **Android Build Support** (marque também *OpenJDK* e *Android SDK & NDK Tools*)
 - **Linux Dedicated Server Build Support** (para o plano B)

Se a versão já estiver instalada: engrenagem ao lado dela → **Add modules**.

4. **ADB (Android Platform Tools):** já vem com o módulo Android do Unity, em
`C:\Program Files\Unity\Hub\Editor\6000.0.39f1\Editor\Data\PlaybackEngines\AndroidPlayer\SDK\platform-tools\adb.exe`
ou baixe separado: <https://developer.android.com/tools/releases/platform-tools>.

1.2 Baixar o projeto do GitHub (com Git LFS)

Abra o **PowerShell** na pasta onde quer o projeto (ex.: `E:\`):

```
git lfs install
git clone https://github.com/danielreiss/Projeto-VR.git
cd Projeto-VR
git checkout claude/awesome-meitner-16iyc
git lfs pull
```

O branch `claude/awesome-meitner-16iyc` tem as mudanças do servidor dedicado (PR #1). Depois que o PR for mergeado na `main`, use `git checkout main` no lugar. Não baixe o ZIP pelo site do GitHub: o ZIP **não** traz os arquivos do Git LFS (texturas viram arquivos de texto pequenos).

Conferir o LFS: `git lfs ls-files | Select-Object -First 5` deve listar arquivos; e as imagens em `Assets` devem ter tamanho real (não 1 KB).

Alternativa — usar sua pasta `E:\testeMultiplayer3.0`: se ela for um repositório git, rode nela `git stash`, `git fetch origin`, `git checkout claude/awesome-meitner-16iyc`, `git lfs pull`, `git stash pop`. Se não for git, copie do PR os arquivos `Assets/Scripts/LanDirectConnect.cs`, `Assets/Editor/ServerBuild.cs` e `.meta`, e ajuste o IP pelo Inspector (passo 1.4). Faça backup antes.

1.3 Abrir no Unity

1. Unity Hub → **Projects** → **Add** → **Add project from disk** → selecione a pasta `Projeto-VR` (a que contém `Assets`, `Packages`, `ProjectSettings`).
2. Abra com a versão **6000.0.39f1**. A primeira importação demora (pode levar bastante tempo) e precisa de internet: o pacote *Meta Movement* é baixado do GitHub via git.

3. Quando terminar, deve aparecer o menu **Build** na barra superior do Unity.

1.4 Conferir o IP do servidor na cena

1. Abra `Assets/Scenes/ConsultorioFinal.unity`.
2. Na Hierarchy, encontre o objeto que tem o componente **Lan Direct Connect** (dica: digite `t:LanDirectConnect` na busca da Hierarchy).
3. Campo **Host Ip Address** = `172.30.22.31`, **Port** = `7777`. Salve (Ctrl+S).

O IP do **Network Manager** → **Unity Transport** pode ficar como está: o `LanDirectConnect` sobrescreve esse valor ao iniciar. Esse mesmo campo é ignorado no build do servidor.

PARTE 2 — Gerar o APK e instalar nos óculos (em casa)

2.1 Ativar o modo desenvolvedor do Quest (uma vez só)

1. Crie/entre numa **organização de desenvolvedor** em `https://developers.meta.com/horizon` (gratuito).
2. No app **Meta Horizon** do celular: **Dispositivos** → **seu Quest** → **Configurações do fone** → **Modo desenvolvedor** → **Ativar**.
3. Reinicie o Quest.

2.2 Gerar o APK

1. No Unity: menu **Build** → **APK Meta Quest (Android)**. O script troca para Android se necessário (a troca de plataforma reimporta e demora).
2. Saída: `Builds\Android\ConsultorioVR.apk`.

Alternativa manual: **File** → **Build Profiles** → **Android** → **Switch Platform**, deixe apenas `ConsultorioFinal` nas cenas e clique **Build**.

2.3 Instalar no Quest

1. Ligue o Quest no PC com cabo USB-C de dados. No headset aparece "**Permitir depuração USB?**" → marque *Sempre permitir* → OK.
2. No PowerShell (na pasta do projeto):

```
adb devices                # deve listar o Quest como "device"
adb install -r Builds\Android\ConsultorioVR.apk
```

Deve aparecer `Success`. Repita para cada óculos (o mesmo APK serve para todos).

Alternativa sem linha de comando: **Meta Quest Developer Hub** → arraste o APK para o dispositivo.

Para abrir: no Quest, **Biblioteca** → **Fontes desconhecidas** (ou "Unknown Sources") → *Consultorio VR*.

2.4 (Recomendado) Teste em casa antes do lab

1. Gere um servidor para Windows: **File** → **Build Profiles** → **Windows Server** (instale o módulo *Windows Dedicated Server Build Support* se pedir), cena `ConsultorioFinal`, **Build**. Rode no PowerShell: `.\ConsultorioServer.exe -logFile -.` Depois volte a plataforma para Android.
2. Descubra o IP do PC: `ipconfig` (ex.: `192.168.0.10`). Libere a porta UDP 7777 no Firewall do Windows se perguntado.
3. Envie esse IP para os óculos (sem rebuild):

```
"192.168.0.10" | Out-File -Encoding ascii server_ip.txt
adb push server_ip.txt /sdcard/Android/data/com.Danie.ConsultorioVR.DriveE/files/server_ip.txt
```

4. Abra o app com o Quest no mesmo Wi-Fi. Se entrou no cenário e o servidor mostra a conexão, está funcionando.
5. **Depois do teste, apague o arquivo** para voltar a usar o IP do servidor do lab:

```
adb shell rm /sdcard/Android/data/com.Danie.ConsultorioVR.DriveE/files/server_ip.txt
```

2.5 Deixar o adb pronto para usar sem USB (para o lab)

No lab a porta USB-C estará com o adaptador Ethernet. Para conseguir logs/enviar arquivos pela rede depois:

```
adb tcpip 5555          # com o USB ainda conectado
# no lab, na mesma rede do Quest:
adb connect IP_DO_QUEST:5555
```

(O modo tcpip é perdido quando o Quest reinicia; é só repetir com USB.)

PARTE 3 — (Plano B) Gerar o build do servidor no Windows

Use se a instalação do Unity no servidor (Parte 4/5) der problema — principalmente a **licença**. O executável Linux gerado no Windows roda no servidor sem Unity instalado.

1. No Unity: **Build → Servidor Linux (Dedicated Server)**.
2. Saída: `Builds\LinuxServer\` (contém `ConsultorioServer.x86_64` e a pasta `ConsultorioServer_Data`).
3. Leve a pasta no notebook/pendrive. No lab, envie com:

```
scp -r .\Builds\LinuxServer admin@172.30.22.31:~/consultorio-build
```

Depois siga direto para a **Parte 6** usando `~/consultorio-build` como pasta.

Recomendação: gere esse build em casa mesmo que pretenda usar a Parte 5. Se der errado no servidor, você não perde o dia.

PARTE 4 — Preparar o servidor Linux (no lab)

4.1 Entrar no servidor

Notebook conectado ao cabo da rede DCC. No PowerShell:

```
ping 172.30.22.31
ssh admin@172.30.22.31      # senha: a enviada pelo professor
```

4.2 Ver o sistema e o espaço em disco

```
cat /etc/os-release      # distribuição (Ubuntu/Debian/Rocky...)
df -h ~                  # precisa de ~25 GB livres para Unity + projeto
nproc; free -h
ip -4 addr                # anote os IPs do servidor (DCC e o da rede 5G!)
```

Importante: no `ip -4 addr`, veja se existe uma interface com IP da rede do roteador 5G. Se os óculos vão pela rede 5G, **esse** é o IP que eles precisam. Se for diferente de `172.30.22.31`, use o `server_ip.txt` (Parte 7.3).

4.3 Instalar git, Git LFS e utilitários

Ubuntu/Debian:

```
sudo apt update
sudo apt install -y git git-lfs tmux wget curl unzip xvfb
git lfs install
```

Rocky/RHEL/Fedora: `sudo dnf install -y git git-lfs tmux wget curl unzip xorg-x11-server-Xvfb`

4.4 Clonar o repositório com o cenário (Git LFS)

```
cd ~
git clone https://github.com/danielreiss/Projeto-VR.git
cd Projeto-VR
git checkout claude/awesome-meitner-16iyc      # ou main, após o merge
git lfs pull
git lfs ls-files | head                        # confere que o LFS baixou
```

Se o repositório for **privado**, o git vai pedir usuário e senha: use seu usuário do GitHub e, como senha, um **Personal Access Token** (GitHub → Settings → Developer settings → Personal access tokens → *Fine-grained*, permissão *Contents: Read* no repositório). Senha normal não funciona.

4.5 Instalar o Unity Editor no servidor (modo sem tela)

Ubuntu/Debian — instalar o Unity Hub:

```
wget -q0 - https://hub.unity3d.com/linux/keys/public | gpg --dearmor | \
  sudo tee /usr/share/keyrings/Unity_Technologies_ApS.gpg > /dev/null
echo "deb [signedby=/usr/share/keyrings/Unity_Technologies_ApS.gpg] https://hub.unity3d.com/linux/repos/deb stable main" | \
  sudo tee /etc/apt/sources.list.d/unityhub.list
sudo apt update
sudo apt install -y unityhub
```

Instalar o Editor 6000.0.39f1 com o módulo de servidor Linux (pelo Hub em modo headless):

```
xvfb-run -a unityhub --no-sandbox --headless install \
  --version 6000.0.39f1 --changeset 15ea7ed0b100 --module linux-server

xvfb-run -a unityhub --no-sandbox --headless editors --installed
```

O editor fica em `~/Unity/Hub/Editor/6000.0.39f1/Editor/Unity`. Guarde em uma variável:

```
UNITY=~/Unity/Hub/Editor/6000.0.39f1/Editor/Unity
ls -l $UNITY
```

4.6 Ativar a licença do Unity

O Editor só roda com licença ativada. Em linha de comando, tente:

```
$UNITY -batchmode -quit -nographics -logFile - \
  -username "SEU_EMAIL_UNITY" -password "SUA_SENHA_UNITY"
```

Atenção: a ativação por linha de comando com `-username/-password` é garantida para licenças **Pro/Educacional com serial** (acrescente `-serial XX-XXXX-...`). Para licença **Personal**, a Unity pode exigir ativação pelo Unity Hub com interface gráfica, o que não é possível só com SSH. Se aparecer erro de licença (*"No valid Unity Editor license found"*), **use o Plano B (Parte 3)** — o resultado é o mesmo. Pergunte também ao DCC/UFGM se há licença educacional (Unity Student/Education Grant).

PARTE 5 — Gerar o build do servidor no próprio servidor

```
cd ~/Projeto-VR
$UNITY -batchmode -quit -nographics -projectPath . \
  -executeMethod ServerBuild.BuildLinuxServer -logFile build.log
tail -n 30 build.log
ls Builds/LinuxServer/
```

- A primeira execução importa o projeto inteiro: pode levar de 20 min a mais de 1 h.
- Sucesso: o log contém [ServerBuild] StandaloneLinux64: Succeeded e existe Builds/LinuxServer/ConsultorioServer.x86_64.
- Erros de compilação: procure error CS no log (grep "error CS" build.log) e me envie.

Atualizar depois de mudar o projeto

```
cd ~/Projeto-VR
git pull
git lfs pull
$UNITY -batchmode -quit -nographics -projectPath . -executeMethod ServerBuild.BuildLinuxSe
rver -logFile build.log
```

Sempre gere APK e servidor a partir do **mesmo commit**. Se a lista de prefabs de rede for diferente, os óculos não conseguem entrar.

PARTE 6 — Rodar o servidor

6.1 Liberar a porta UDP 7777

```
sudo ufw status # Ubuntu
sudo ufw allow 7777/udp

sudo firewall-cmd --add-port=7777/udp --permanent && sudo firewall-cmd --reload # Rocky/RHEL
```

6.2 Iniciar dentro do tmux (continua rodando se o SSH cair)

```
tmux new -s vr
cd ~/Projeto-VR/Builds/LinuxServer # ou ~/consultorio-build (Plano B)
chmod +x ConsultorioServer.x86_64
./ConsultorioServer.x86_64 -logFile -
```

Procure no log a linha:

```
--- LAN: Iniciando como SERVIDOR DEDICADO na porta 7777 ---
```

- Sair do tmux sem parar o servidor: **Ctrl+B**, depois **D**.
- Voltar: `tmux attach -t vr`. Parar o servidor: **Ctrl+C** dentro do tmux.
- Conferir se está escutando (em outro terminal): `ss -ulpn | grep 7777`

Avisos de XR/OVR/áudio no log são esperados (o servidor não tem óculos nem placa de som). O que importa é a linha acima.

6.3 Argumentos opcionais

Argumento	Efeito
<code>-port 7777</code>	Troca a porta (os óculos precisam usar a mesma — rebuild ou ajuste na cena)
<code>-server</code>	Força modo servidor (já é o padrão no build Dedicated Server)
<code>-logFile server.log</code>	Grava o log em arquivo em vez da tela

PARTE 7 — Conectar os óculos e entrar no cenário

7.1 Ligar o Quest na rede

1. Adaptador **USB-C → Ethernet** no Quest (de preferência com entrada de carga USB-C PD, para não descarregar).
2. Cabo do adaptador no **roteador 5G** (ou na rede onde o servidor é alcançável).
3. No Quest: **Configurações → Wi-Fi → desligar**, para garantir que o tráfego vá pelo cabo. Em Configurações deve aparecer a conexão Ethernet com IP.

7.2 Entrar

1. Confirme que o servidor está rodando (Parte 6).
2. No Quest: **Biblioteca → Fontes desconhecidas → Consultorio VR**.
3. O app conecta sozinho (e continua tentando até o servidor responder). No log do servidor aparece a conexão do cliente. Repita em cada óculos — todos entram na mesma sala.

Reconexão automática: se o servidor ainda não estiver no ar, a rede demorar a ligar ou a conexão cair, o app tenta de novo sozinho a cada 3 segundos (no log: LAN: sem conexão ... Nova tentativa). Não precisa reabrir o app. O intervalo pode ser ajustado no componente **Lan Direct Connect** (*Reconnect Interval*).

7.3 Se o IP do servidor for outro nessa rede

Não precisa refazer o APK. Pelo notebook com adb (USB ou `adb connect IP_D0_QUEST:5555`):

```
"NOVO_IP" | Out-File -Encoding ascii server_ip.txt
adb push server_ip.txt /sdcard/Android/data/com.Danie.ConsultorioVR.DriveE/files/server_ip.txt
```

Prioridade do IP: argumento `-ip` → arquivo `server_ip.txt` → campo da cena.

PARTE 8 — Problemas comuns

Sintoma	Causa provável / solução
Texturas rosas/erradas ou arquivos de 1 KB	Git LFS não baixou: <code>git lfs install</code> + <code>git lfs pull</code> .
Unity não resolve pacote <i>com.meta.xr.sdk.movement</i>	Git não instalado/no PATH, ou sem internet. Instale o git e reabra.
Menu Build não aparece	Não está no branch certo ou há erro de compilação no Console.
<code>adb devices</code> vazio / <i>unauthorized</i>	Aceite "Permitir depuração USB" no headset; use cabo de dados; modo desenvolvedor ativo.
"No valid Unity Editor license found" no servidor	Licença Personal não ativa via SSH → Plano B (Parte 3).
<code>Permission denied</code> ao rodar o servidor	<code>chmod +x ConsultorioServer.x86_64</code>
Óculos abrem o cenário mas não aparece ninguém / não conecta	Servidor parado; IP errado (Parte 7.3); porta UDP 7777 bloqueada; óculos em rede que não alcança o servidor. Teste com o log: <code>adb logcat -s Unity</code> e procure LAN: .
Servidor some ao fechar o SSH	Rode dentro do <code>tmux</code> (Parte 6.2).
Voz não chega	Permita o microfone ao app no Quest (Configurações → Apps → Permissões).

Checklist

Em casa

- ☐ Git, Unity Hub, Unity 6000.0.39f1 + Android + Linux Dedicated Server instalados
- ☐ Repositório clonado, branch certo, `git lfs pull` feito
- ☐ IP `172.30.22.31` conferido no LanDirectConnect
- ☐ APK gerado e instalado em todos os óculos
- ☐ Teste em casa com servidor no PC (e `server_ip.txt` apagado depois)
- ☐ `adb tcpip 5555` feito
- ☐ Build do servidor Linux (Plano B) gerado e copiado para o notebook
- ☐ Adaptador USB-C → Ethernet (com carga) e cabo

No lab

- ☐ SSH no servidor ok; `ip -4 addr` anotado (IP visto pela rede 5G)
- ☐ git + git-lfs instalados, repositório clonado e `git lfs pull`
- ☐ Build do servidor gerado (Parte 5) ou enviado por scp (Plano B)
- ☐ Porta UDP 7777 liberada
- ☐ Servidor rodando no `tmux` com "SERVIDOR DEDICADO" no log
- ☐ Quest no cabo, Wi-Fi desligado, app aberto → dentro do cenário